

Clutch Time Media

Dossier de soutenance - Partie SQL relationnelle

Projet	Blog basket Clutch Time Media
Bloc vise	Backend securise et base de donnees relationnelle
Base	MySQL - modele relationnel
Backend	Node.js / Express
Deploiement	GitHub Pages + Railway + MySQL Railway

Chaine finale validee

GitHub Pages -> Backend Railway -> MySQL Railway

URLs principales

<https://kengni-tayo-albert.github.io/clutch-time-media/>

<https://clutch-time-media-production.up.railway.app/api/health>

<https://clutch-time-media-production.up.railway.app/api/articles>

Les 5 grandes etapes

1	Modelisation	MCD Looping avec article, auteur, categorie, commentaire, tag.
2	Base SQL	Creation de la base blog, tables, cles primaires et cles etrangeres.
3	Tests SQL et CRUD	Insertion, lecture, modification, suppression et requetes jury.
4	Backend	API Node.js / Express entre le front et MySQL.
5	Deploiement	GitHub Pages, Railway backend, MySQL Railway et verification finale.

Objectif du document

Ce document explique la partie SQL relationnelle du projet Clutch Time Media.

Le but est de montrer au jury que le projet ne se limite plus a un site front-end ou a un fichier JSON. Il utilise maintenant une base de donnees relationnelle MySQL, un backend Node.js / Express et une API pour faire circuler les donnees.

Chaine finale du projet :

GitHub Pages -> Backend Railway -> MySQL Railway

1. Modelisation de la base

Avant de creer les tables, la base a eteensee avec un MCD dans Looping.

Les entites principales du blog sont :

article
auteur
categorie
commentaire
tag

Les relations principales sont :

Un auteur peut ecrire plusieurs articles.
Un article appartient a une categorie.
Un article peut recevoir plusieurs commentaires.
Un commentaire est redige par un auteur.
Un article peut avoir plusieurs tags.
Un tag peut etre associe a plusieurs articles.

Exemple concret tire du blog :

Article : OKC champion NBA
Categorie : Teams
Auteur : Tom Tatum
Tags : NBA, Champion, Oklahoma City

Discours jury :

J'ai commence par la modelisation, car une base relationnelle doit etre structuree avant d'etre codee. Avec Looping, j'ai identifie les entites du blog, puis les relations entre elles. Cela permet d'eviter de tout stocker au meme endroit et de montrer clairement les liens entre les articles, les auteurs, les categories, les commentaires et les tags.

2. Creation de la base SQL

La base MySQL s'appelle :

blog

Elle contient les tables :

categorie
auteur
article
commentaire
tag
taguer

La table `taguer` est une table d'association. Elle sert a gerer la relation plusieurs-a-plusieurs entre `article` et `tag`.

Exemple :

Un article peut avoir plusieurs tags.
Un tag peut etre utilise par plusieurs articles.

Les scripts SQL sont ranges dans cet ordre :

01_creation_base_blog.sql
02_creation_tables_blog.sql
03_peuplement_blog.sql
04_crud_blog.sql
05_requetes_jury_blog.sql
06_verification_finale_blog.sql

Discours jury :

Dans DBeaver, j'ai cree la base blog, puis les tables relationnelles. Chaque table possede une cle primaire. Les relations sont faites avec des cle etrangeres. Par exemple, article.id_auteur pointe vers auteur.id_auteur, et commentaire.id_article pointe vers article.id_article.

3. Tests SQL et CRUD

Le CRUD signifie :

Create : creer une donnee
Read : lire une donnee
Update : modifier une donnee
Delete : supprimer une donnee

Exemples testes dans le projet :

Creer un article depuis le formulaire du site.
Lire la liste des articles.
Modifier un article avec une requete UPDATE.
Supprimer un article avec une requete DELETE.
Creer un commentaire sur un article.
Lire les commentaires relies a un article.

Exemple SQL de lecture avec jointures :

```
SELECT
  ar.id_article,
  ar.titre,
  ar.resumer,
  au.nom_auteur,
  ca.nom_categorie
FROM article ar
JOIN auteur au ON ar.id_auteur = au.id_auteur
JOIN categorie ca ON ar.id_categorie = ca.id_categorie
ORDER BY ar.id_article DESC;
```

Exemple tire du blog :

Article cree en production : Test production SQL
Auteur : Alberto TAYO
Categorie : Players
Commentaire ajoute : Excellent
Auteur du commentaire : Oscar

Discours jury :

J'ai teste les operations CRUD pour montrer que la base n'est pas seulement theorique. Les donnees peuvent etre creees, lues, modifiees et supprimees. Les requetes de lecture utilisent aussi des jointures pour recuperer les informations venant de plusieurs tables.

4. Connexion du site avec un backend

Avant, le site pouvait afficher des donnees depuis un fichier JSON.

Maintenant, le fonctionnement est :

Page HTML / JavaScript -> API Express -> Requetes SQL -> MySQL

Le backend contient des routes API, par exemple :

```
GET    /api/health
GET    /api/articles
GET    /api/articles/:id
POST   /api/articles
PUT    /api/articles/:id
DELETE /api/articles/:id
GET    /api/articles/:id/comments
POST   /api/articles/:id/comments
```

Exemple :

Quand la page articles.html charge les articles, elle appelle /api/articles.
Le backend execute une requete SQL.
MySQL renvoie les articles.
Le backend transforme le resultat en JSON.
Le front affiche les cartes d'articles.

Discours jury :

Le backend sert d'intermediaire entre le site et la base SQL. Le front ne se connecte pas directement a MySQL. Il appelle une API. Cela permet de mieux organiser le projet, de proteger les acces a la base et de centraliser les requetes SQL cote serveur.

5. Deploiement et verification finale

Le projet fonctionne maintenant en ligne avec :

Front : GitHub Pages
Backend : Railway
Base SQL : MySQL Railway

URLs finales :

Site public :
<https://kengni-tayo-albert.github.io/clutch-time-media/>

Page articles :
<https://kengni-tayo-albert.github.io/clutch-time-media/assets/pages/articles.html>

Backend Railway :
`https://clutch-time-media-production.up.railway.app`

Health check :
`https://clutch-time-media-production.up.railway.app/api/health`

Articles :
`https://clutch-time-media-production.up.railway.app/api/articles`

Commentaires de l'article 5 :
`https://clutch-time-media-production.up.railway.app/api/articles/5/comments`

Preuves validees :

`/api/health` renvoie `{"status":"ok","database":"blog"}`.
`/api/articles` renvoie les articles stockes dans MySQL Railway.
`/api/articles/5/comments` renvoie le commentaire cree depuis le site.

Discours jury :

Pour le deploiement, je ne pouvais pas utiliser la base MySQL locale en localhost, car localhost fonctionne seulement sur mon ordinateur. J'ai donc cree une base MySQL distante sur Railway, puis j'ai deploye le backend Express sur Railway. Le site public GitHub Pages appelle maintenant cette API en ligne.

Difference avec MongoDB

Avec MongoDB, les donnees sont stockees sous forme de documents.

Exemple MongoDB :

Un article peut contenir directement son auteur, ses tags et ses commentaires dans le meme document.

Avec MySQL, les donnees sont separees dans des tables reliees.

Exemple MySQL :

```
article.id_auteur pointe vers auteur.id_auteur
article.id_categorie pointe vers categorie.id_categorie
commentaire.id_article pointe vers article.id_article
taguer relie article et tag
```

Phrase simple :

MongoDB est oriente documents, alors que MySQL est relationnel. Dans ce projet, MySQL me permet de montrer les relations entre les articles, les auteurs, les categories, les commentaires et les tags.

Securite minimale

Elements mis en place :

```
Le mot de passe MySQL n'est pas ecrit directement dans le code.
Le fichier .env est ignore par Git.
Les requetes SQL utilisent mysql2 avec des parametres.
Les champs envoyes par les formulaires sont verifies cote backend.
Les identifiants dans les routes sont controles.
Le backend limite les origines autorisees avec CORS.
```

Discours complet court

Pour ce projet, je suis parti du site Clutch Time Media, qui etait principalement front-end. Pour couvrir le bloc SQL relationnel, j'ai commence par modeliser une base de donnees de blog avec Looping.

J'ai ensuite cree la base MySQL blog dans DBeaver, avec plusieurs tables : article, auteur, categorie, commentaire, tag et taguer. Chaque table a une cle primaire, et les relations sont gerees avec des cles etrangeres.

J'ai ajoute des donnees du vrai blog, puis j'ai teste les requetes SQL demandees : affichage des articles, recherche par mot, articles recents, commentaires par article, nombre de tags par article et moyennes de temps de lecture.

Ensuite, j'ai developpe un backend Node.js / Express. Le front appelle ce backend avec fetch, et le backend execute les requetes SQL vers MySQL. Cela permet au site de lire les articles, creer des articles et creer des commentaires.

Enfin, j'ai deploye le backend et la base MySQL sur Railway, puis j'ai connecte le site public GitHub Pages a cette API. La chaine finale est donc : GitHub Pages, API Railway, MySQL Railway.

Conclusion

Cette partie du projet montre :

La modelisation d'une base relationnelle.
La creation d'une base MySQL.
L'utilisation de cles primaires et cles etrangeres.
Des requetes SQL avec jointures.
Des operations CRUD.
Un backend Express connecte a MySQL.
Un deploiement public avec Railway.
Une documentation claire pour la soutenance.

Conclusion rapide a dire

*J'ai transforme Clutch Time Media en projet connecte a une base SQL relationnelle.
Le jury peut verifier la modelisation, les tables, les requetes, l'API, le CRUD et le deploiement en ligne.*